Sistemas Operativos

Nombre y apellido: _____

Departamento de Computación – FCEyN – UBA

Cant. hojas: 5

Segundo cuatrimestre de 2025

Segundo parcial – 2do cuatrimestre de 2025

1	2	3	Nota
B-	B-	B	(A)

- Cada ejercicio debe realizarse en **hojas separadas y numeradas**. Debe identificarse cada hoja con **nombre, apellido, LU y número de orden**. Hojas sin identificar no serán corregidas. Completar en la hoja de enunciado la cantidad total de hojas entregadas y entregarla junto al examen, la misma **no** se incluye en la cantidad total de hojas entregadas.
- La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, sin retirar el examen corregido del aula.
- Cada ejercicio se califica con **Bien**, **Regular** o **Mal**. La división de los ejercicios en incisos es meramente orientativa y cada ejercicio se califica globalmente. El parcial se aprueba con un mínimo de 2 ejercicios Bien y 1 Regular.

Ejercicio 1.

Se tiene un sistema de archivos `ext2`. Se pide:

a) Implementar una función:

```
int count_files_with_extension(char *dir, char *extension, int min_size);
```

que tome el path de un directorio, una extensión (por ejemplo, ".txt") y una cantidad mínima de bytes, y retorne la cantidad total de archivos regulares (no directorios ni enlaces) que:

- terminen con la extensión indicada, y
- tengan un tamaño mayor o igual a `min_size`.

La búsqueda deberá considerar también todos los archivos ubicados en los subdirectorios de `dir` (en cualquier nivel).

Se cuenta con las estructuras vistas para representar inodos y entradas de directorio.

Además, se dispone de las siguientes funciones auxiliares:

```
struct Ext2FSInode *Ext2FS::inode_for_path(const char *path);
// dado un path, devuelve su inodo

void Ext2FS::read_block(unsigned int block_address, unsigned char *buffer);
// dada una dirección de bloque y un buffer, carga el bloque en el buffer

unsigned int Ext2FS::get_block_address(struct Ext2FSInode *inode, unsigned int block_number);
// dados un inodo y un número de bloque, devuelve la dirección del bloque de datos

struct Ext2FSInode *Ext2FS::load_inode(unsigned int inode_number);
// dado un número de inodo, busca el inodo en el grupo y lo devuelve

char* get_extension_from_filename(char* filename);
// dado un nombre de archivo, devuelve su extensión
```

b) ¿Cómo debería modificarse la función si se quisiera contabilizar también los archivos accedidos (no modificados) en los últimos 7 días? Indicar qué campos del inodo utilizaría y cuál sería la condición a verificar.

c) ¿Qué problemas podrían surgir si el recorrido incluye enlaces simbólicos que apuntan a directorios?

Ejercicio 2.

Se desea implementar un **driver para un sensor de proximidad con un LED indicador**. El sensor mide distancia en centímetros y genera una interrupción cada vez que realiza una nueva medición, es decir, cuando el valor de distancia se actualiza. El LED debe encenderse o apagarse según la proximidad detectada.

El dispositivo cuenta con los siguientes registros de E/S mapeados en memoria:

PROX_DIST	Entero que contiene la última distancia medida, expresada en centímetros.
LED_COLOR	Entero que indica el color actual del LED (valores predefinidos: LED_OFF, LED_GREEN, LED_RED).
LED_CTRL	Registro de control: al escribir LED_UPDATE, aplica el color actualmente configurado en LED_COLOR.
IRQ_SENSE_FINISHED	Señal de interrupción que se genera cada vez que hay una nueva medición disponible en PROX_DIST.

El sistema operativo notificará al driver cuando ocurra la interrupción IRQ_SENSE_FINISHED. El driver deberá registrar el evento y poner a disposición del espacio de usuario los datos del último valor medido por el sensor.

a) Implementar las funciones necesarias del driver (incluyendo posibles Rutinas de Atención de Interrupciones) para permitir el acceso de un usuario a las mediciones del sensor y al control (cambio de color) del LED. La lectura deberá retornar el valor de la *próxima* medición a partir de ese momento.

b) Escribir el pseudocódigo de un programa de usuario que solicite un valor numérico por consola y lo compare con la distancia medida por el sensor. Si la distancia medida es menor o igual al umbral ingresado, se deberá encender el LED de color rojo; en caso contrario se deberá encender el LED de color verde.

c) Explicar brevemente qué problemas (desde el punto de vista de la sincronización) podría tener el driver para manejar lecturas/escrituras desde múltiples programas de usuario, y qué debería cambiarse o agregarse al driver para resolverlos.

Ejercicio 3.

El siguiente fragmento en C, que se ejecuta con privilegios elevados, intenta mostrar el contenido de un archivo indicado por el usuario usando `cat`:

```
#define BUF_SIZE 1024
void wrapper_cat(const char * filename) {
    char cmd[BUF_SIZE];
    snprintf(cmd, BUF_SIZE, "cat %s", filename);
    system(cmd);
}
```

a) Mostrar un valor de la cadena `filename` que, además de mostrar el archivo indicado, ejecute `rm -rf /tmp/important`.

b) Se modificó la línea de `snprintf` por:

```
snprintf(cmd, BUF_SIZE, "cat \"%s\"", filename);
```

Mostrar por qué esta modificación no elimina la vulnerabilidad y proporcionar un ejemplo de `filename` que la explote.

Ej 1 : 1/2

Ejercicio 1 (a) Aprovechamos las funciones que nos dan para inicialmente conseguir el inodo del directorio. Una vez hecho eso la idea es aprovechar las estructuras para recursivamente ir recorriendo los subdirectorios y en el camino contar los archivos que cumplan la condición pedida.

```
int cfwe(char* dir, char* extension, int mim_size) {
    return procesar(inode_for_path(dir), extension, mim_size);
}

int procesar(EXT2FSinode * inode_dir, char* extension, int mim_size) {
    char data[2* BLOCK_SIZE]; int offSET = BLOCK_SIZE; int total = 0;
    int contador = 0;                                          (*3)
    for (int i = 0; total < inode_dir->size; i++) {                (*)1
        offset -= BLOCK_SIZE;
                (*)4
        obtener_bloque(inode_dir, i, data);
        obtener_bloque(inode_dir, i+1, data + BLOCK_SIZE); Puede no existir
        while (offset < BLOCK_SIZE) {  (*)2  Te podés pasar de inode_dir->size
            EXT2FSDIRENTRY * entry = data + offset;
            if (strcmp(entry->name, ".")==0 || strcmp(entry->name, "..")==0) {
                offset += entry->record_length;
                continue;
            }
            if (entry->file_type=='d') {
                EXT2FSINODE * inode_subdir = load_inode(entry->inode);
                contador += procesar(inode_subdir, extension, mim_size);
            } else if (entry->file_type =='r') {
                (*)5 if(strcmp(get_ext_from_filename(entry->name), extension)==0) {
                    EXT2FSINODE * inode_file = load_inode(entry->inode);
                    if(inode_file->size >= mim_size) {
                        contador ++;
                    }
                }
            }
        }
    }
```

sigo →

```
        offset += record_length;
        } // fin while
    total += offset   // cant. Bytes leidos.
} // fin for

return contador
} // fin fn
```

El objetivo de (*1) es iterar por todos los bloques de datos del inodo del directorio, mientras no hayamos leido la cant necesarios de Bytes (total < inode->size);

El objetivo de (*2) es salvar el caso de una entrada partida en dos bloques (ya que pueden tener long. variable). La idea es leer entradas mientras no pasemos el tamaño de un bloque pero si "hay que pasar un poquito" no hay problema dado que tenemos en memoria 2 bloques contiguos. El indice en este caso se arregla con aritmética de punteros en (*4). Una "optimización" que podria hacerse es no leer en todas las iteraciones los bloques i e i+1 : siempre el bloque i+1 ya está leido, salvo si i=0.

En (*3) se ve un uso de BLOCK_SIZE: este dato puede ser sacado facilmente del superbloque. En la variable contador guardamos el valor que nos interesa retornar. Una función auxiliar usada fue obtener_bloque:

```
void obtener_bloque (EXT2FSINODE* inode, int block, char* buf) {
    int addr = get_block_address (inode, block);
    read_block (addr, buf);
}
//
```

(b) Si quisiera contabilizar también los archivos accedidos en los últimos 7 días, bastaría obtener el dato del inodo del archivo que se esté evaluando en la iteración (sus metadatos). No recuerdo exactamente el nombre pero podría ser algo como access_time. Hay otros atributos de tiempo que se pueden combinar para armar una condición más compleja.
Como quiero contabilizar TAMBIÉN los archivos accedidos la condición en (*)s sería similar a:

```
EXT2FSINODE* inode_file = load_inode(entry->inode);
if(strcmp(...) == 0 ∧ inode_file->size ≥ min_size) ||
    nueva_condicion(entry->access_time)) {
  contador++;
}
```

Donde nueva condicion chequearía lo pedido.

(c) Si el recorrido incluye enlaces simbólicos a directorios lo que podría pasar en términos funcionales es que se contabilicen archivos que no están "directamente" en el directorio especificado. Inclusive podrían estar en otro FSystem. También podría pasar que se pierda el elemento al cual se apuntaban los links (si se elimina por ejemplo): en este caso a priori no podemos manejar el caso (los inodos solo cuentan los hard links a ellos).

Falta mencionar problema principal: recursión infinita

Ej 2: 1/2

Ejercicio 2 (a)

Registros	Variables	Interrupciones
PROX_DIST E/S	Sem hay_medicion	IRQ_SENSE_FINISHED
LED_COLOR E/S		
LED_CTRL E/S.		

Dado que el punto (c). pide analizar específicamente el caso de "multiples usuarios", voy a asumir que hay uno solo para este punto. (fue preguntado). OK

Implementemos primero la lectura teniendo en cuenta que el usuario debe estar bloqueado hasta tener una nueva medición.

```
int driver_read(int* data) {
    hay_medicion.wait();   * No aseguran que sea el próximo ¿y si hubo int. antes?
    int medicion = IN(PROX_DIST);
    copy_to_user(data, &medicion, sizeof(medicion));
    return sizeof(medicion);
}
```

La idea es que el usuario espera mediante un semaforo que haya una nueva medición, ese dato es leido del registro y copiado a la memoria del usuario.

Implementemos a continuación la funcion para la escritura que le permitirá al usuario controlar el LED. [illegible]

typedef [illegible]

[illegible]

[illegible].

Sigo →.

```
int driver_write (int* data) {
    // Según la API que estoy definiendo, en *data espero
    // LED_OFF, LED_GREEN, LED_RED.
    int color;
    copy_from_user (&color, data, sizeof (data));
    OUT (LED_COLOR, color);
    OUT (LED_CTRL, LED_UPDATE); (*)
    return sizeof (data);
}
```

(*) Me hace un poco de ruido: Es el único valor posible para LED_CTRL, en teoría no cambia nunca: Asumo que después del UPDATE el controlador limpia el registro.

Ahora las funciones que correrán al cargar y descargar el módulo.

```
int driver_init() {
    hay_medicion = Sem(0);
    request_irq (IRQ_SENSE_FINISHED, &handler);
}

int driver_remove() {
    hay_medicion.destroy();
    free_irq (IRQ_SENSE_FINISHED);
}

void handler() {
    // en este punto tenemos una nueva medición
    // dejamos pasar al usuario.
    hay_medicion.signal();
}
```

(b) →

(b)
```
void programa_usuario() {
    int umbral;
    printf("umbral: ");
    scanf("%d", &umbral); // Se lee una sola vez siguiendo el comentario hecho en el parcial.
    int fd_dev = open("/dev/sensor");
    while(1) {
        int medicion;
        read(fd_dev, &medicion); // bloqueante.
        if (medicion ≤ umbral) {
            write(fd_dev, LED_RED);
        } else {
            write(fd_dev, LED_GREEN);
        }
    }
    close(fd_dev); // no deberia llegar nunca, pero hay que cerrarlo y queria mostrarlo.
}
```

(c). Problemas desde el punto de vista de la sincronización.

Lecturas: Múltiples usuarios podrían hacer una lectura antes de la próxima interrupción, entonces, al llegar una nueva medición un solo hay_medicion.signal() no bastaría para que todas las lecturas se desbloqueen y obtengan el dato.

Solución: Se podría agregar un esquema de sincronización tipo "molinete" como para el primer parcial o bien registrar cuantos usuarios hay queriendo leer y darles el .signal a todos ellos: hay_medicion.signal(N); (es un for por dentro).

Sigo →

Escrituras: Múltiples usuarios podrían querer cambiar el LED de color (dependiendo del umbral de c/uno). Estos pueden ser incompatibles (ROJO vs VERDE). Esto es un problema más funcional o de especificación más que de implementación: Podría especificarse en la API una forma de resolver este conflicto.

B

Ejercicio 3 (a) El problema de la función wrapper_cat deriva del contenido de filename, mediante el comando snprintf se arma un string que en principio parece ser un comando "cat" para ser ejecutado por system(...). Mediante filename un usuario mal intencionado podría darnos como input:

archivo.txt; rm -rf /tmp/important

esto incluso podría ser un --help.

Eliminando así el archivo en cuestión: es una inyección de comandos que puede derivar en diferentes problemas (como el que vimos o acceso a datos privilegiados pues se ejecuta con privilegios elevados) y una solución está en sanitizar filename: validar que no hayan caracteres que no esperamos, como "-" ó ";".

(b) Si el usuario enviara el siguiente input:

archivo.txt\"; rm -rf /tmp/important; echo \"

Se estarían ejecutando tres comandos siendo el rm el conflictivo. Esto sucede porque el string que ejecuta system(...) es:

cat \"\"; rm -rf /tmp/important; echo \"\"

Una vez más, la solución estaría en sanitizar el input.

Agrego: printf es como un pequeño intérprete de comandos. Cuando el input puede tener estos comandos, entonces corremos riesgo de ~~que~~ sufrir algún problema. Ejemplo: con %x podrían leerse valores del stack y con %n podrían alterarse valores en el stack (como la dir de retorno). Si bien esto lo vimos en la práctica, lo menciono porque tal vez snprintf puede sufrir de algo similar con por ejemplo filename: %x...%x...